DATA CLEANING & VISUALIZATION PROJECT

Sales Dataset Analysis Using Python

Objective

To clean a raw sales dataset, handle missing values, outliers and duplicate records, process the data using Pandas, and create visualizations using Matplotlib and Seaborn to identify useful business insights.

Tools Used

Python, Pandas, NumPy, Matplotlib, Seaborn and Jupyter Notebook.

1. Dataset Description

Column	Description	Data Type
Order_ID	Unique order number	Integer
Order_Date	Date of the order	Date
Product	Product name	Text
Category	Product category	Text
Quantity	Number of units sold	Integer
Sales	Sales amount	Float
Profit	Profit earned	Float
Region	Sales region	Text

2. Raw Data Quality Problems

The raw dataset is checked for common data-quality issues:

• **Missing values:** Empty values may occur in Sales, Profit, Category or Region.

• **Duplicates:** The same order may appear more than once.

• **Outliers:** Very large Sales or Profit values may distort averages and charts.

• **Incorrect data types:** Order_Date may initially be stored as text.

• **Inconsistent text:** Category or Region values may contain extra spaces or different capitalization.

3. Python Data Cleaning Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("sales_data.csv")

print(df.head())
print(df.info())
print(df.isnull().sum())

# Remove duplicates
df = df.drop_duplicates()

# Clean text columns
text_cols = ["Product", "Category", "Region"]
for col in text_cols:
    df[col] = df[col].astype(str).str.strip()

# Convert data types
df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")
df["Profit"] = pd.to_numeric(df["Profit"], errors="coerce")

# Fill missing numeric values with median
for col in ["Quantity", "Sales", "Profit"]:
    df[col] = df[col].fillna(df[col].median())

# Fill missing categorical values with mode
for col in ["Product", "Category", "Region"]:
    df[col] = df[col].fillna(df[col].mode()[0])

# Remove invalid dates
df = df.dropna(subset=["Order_Date"])

print(df.isnull().sum())
```

4. Outlier Detection and Treatment

The Interquartile Range (IQR) method detects unusually high or low values. Values below Q1 – 1.5×IQR or above Q3 + 1.5×IQR are treated as outliers. Clipping is used so that genuine large orders are not automatically deleted.

```
def cap_outliers(data, column):
    q1 = data[column].quantile(0.25)
    q3 = data[column].quantile(0.75)
    iqr = q3 - q1

    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr

    data[column] = data[column].clip(lower, upper)
    return data

for col in ["Sales", "Profit", "Quantity"]:
    df = cap_outliers(df, col)

df.to_csv("cleaned_sales_data.csv", index=False)
```

5. Exploratory Data Analysis

After cleaning, the data can be summarized to understand sales, profit, products and regions.

```
print(df.describe())

print("Total Sales:", df["Sales"].sum())
print("Total Profit:", df["Profit"].sum())

category_summary = df.groupby("Category")[["Sales", "Profit"]].sum()
print(category_summary)

region_summary = df.groupby("Region")[["Sales", "Profit"]].sum()
print(region_summary)
```

6. Data Visualization

```
# Sales by Category
category_sales = df.groupby("Category")["Sales"].sum().sort_values(ascending=False)

plt.figure(figsize=(8,5))
category_sales.plot(kind="bar")
plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# Profit by Region
region_profit = df.groupby("Region")["Profit"].sum().sort_values(ascending=False)

plt.figure(figsize=(8,5))
region_profit.plot(kind="bar")
plt.title("Profit by Region")
plt.xlabel("Region")
plt.ylabel("Profit")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# Sales distribution
plt.figure(figsize=(8,5))
sns.histplot(df["Sales"], bins=20, kde=True)
plt.title("Distribution of Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# Sales vs Profit
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x="Sales", y="Profit")
plt.title("Sales vs Profit")
```

```
plt.tight_layout()
plt.show()
```

7. Expected Findings

The exact findings depend on the dataset used. After running the program, report the categories and regions with the highest sales/profit, the presence of unusual transactions, and whether higher sales generally correspond to higher profit. Use the actual values produced by the cleaned dataset.

Analysis	Finding to report
Data quality	Number of missing values and duplicates before and after cleaning.
Category	Category with the highest total sales.
Region	Region with the highest total profit.
Outliers	Number of outliers detected and how they were treated.
Relationship	Whether Sales and Profit show a positive, negative or weak relationship.

8. Conclusion

This project demonstrates a complete data-analysis workflow. The raw sales data is inspected and cleaned by handling missing values, duplicates, incorrect data types and outliers. Pandas is then used to summarize the cleaned data, while Matplotlib and Seaborn are used to create charts. The visualizations make it easier to understand category performance, regional profit, sales distribution and the relationship between sales and profit.

9. Learning Outcomes

• Learned how to inspect and clean a raw dataset using Pandas.

• Learned how to handle missing values and duplicate records.

• Learned how to detect and treat outliers using the IQR method.

• Learned how to group and summarize data for analysis.

• Learned how to create bar charts, histograms and scatter plots.

• Learned how to communicate findings through a visual report.